

1. Uvod

Distribuirani sistemi danas su okosnica modernog digitalnog sveta, od blockchain mreža i digitalnih valuta centralnih banaka do sistema za upravljanje integritetom baza podataka. U srcu ovih sistema nalazi se problem konsenzusa: kako grupa nezavisnih računarskih čvorova (replika) može da se dogovori o jedinstvenom redosledu transakcija, čak i kada neki od njih ne rade ispravno ili su zlonamerni?

Vizantijski tolerantni (Byzantine Fault Tolerant – BFT) protokoli pružaju upravo ovu apstrakciju, centralizovan, pouzdan i uvek dostupan server, uprkos tome što pojedine replike mogu da odstupe od protokola.

Postojeći BFT protokoli suočavaju se sa fundamentalnim kompromisom: oni koji su brzi u idealnim uslovima sporo se oporavljaju od kvarova, dok oni koji su robusni na kvarove imaju veće latencije čak i kada je mreža stabilna. Upravo ovaj problem rešava Autobahn – novi BFT protokol predstavljen na SOSP 2024. konferenciji.

2. Motivacija: Problem blipova i hangovera

2.1 Parcijalna sinhronost u praksi

Teorijski model parcijalne sinhronosti pretpostavlja da nakon nekog "magičnog" trenutka (Global Stabilization Time – GST) mreža postaje trajno sinhrona. U praksi, međutim, ovo je fikcija. Realne deploymente karakterišu:

- Nepredvidiva kašnjenja u WAN mrežama koja mogu varirati za red veličine
- Asimetrične mrežne veze i heterogeni hardverski resursi
- Napadi uskraćivanjem usluge (DDoS) i preusmeravanjem ruta
- Namerno usporavanje od strane vizantijskih replika

Ovi faktori dovode do periodičnih prekida napretka, blipova.

2.2 Hangoveri – tihi ubice performansi

Ključni doprinos Autobahna jeste formalizacija pojma hangovera:

> *Hangover je svako degradiranje performansi uzrokovano blipom koje se nastavlja i nakon povratka dobrog intervala.*

Hangoveri utiču i na propusnu moć i na latenciju. Tokom blipa, transakcije se gomilaju u redu čekanja. Kada se mreža oporavi, protokolu treba vreme da "razreši" ovaj zaostatak a to vreme može biti znatno duže od samog trajanja blipa.

2.3 Postojeći pristupi i njihovi nedostaci

Tradicionalni BFT protokoli (PBFT, HotStuff) optimizuju performanse za dobre intervale, ali su podložni hangoverima. Razlog je tesna sprega diseminacije podataka i konsenzusa: lider istovremeno šalje podatke i predlaže njihov redosled. Kada konsenzus zastane, zastaje i diseminacija i stvarajući zaostatak srazmeran trajanju blipa.

DAG-based protokoli (Bullshark, Narwhal) odvajaju diseminaciju od konsenzusa i oporavljaju se gracioznije, ali su zato spori u dobrim intervalima zahtevaju do 12 krugova razmene poruka za potvrdu transakcije.

Autobahn donosi najbolje iz oba sveta: nisku latenciju tradicionalnih protokola i robusnost DAG-based pristupa.

3. Opis Autobahn algoritma

3.1 Arhitektura: Dva nezavisna sloja

Autobahn razdvaja sistem na dva logički nezavisna sloja:

1. Sloj diseminacije podataka – asinhron, radi kontinuirano u pozadini, nezavisno od stanja konsenzusa. Njegov jedini cilj je da osigura dostupnost podataka.
2. Sloj konsenzusa – parcijalno sinhron, pokreće se tek kada postoje dokazi o dostupnosti podataka. Njegov zadatak je da uspostavi jedinstveni redosled.

Ovo razdvajanje omogućava da diseminacija napreduje tempom mreže čak i tokom blipova, dok konsenzus može da se fokusira isključivo na redosled.

3.2 CAR – osnovna jedinica podataka

Osnovna jedinica u Autobahnu je CAR (Certification of Available Request) – struktura koja sadrži:

```
struct CAR {  
    id: redni broj u traci  
    lane_id: identifikator validatora
```

```
prev_hash: hash prethodnog CAR-a u istoj traci
batch_hash: hash batch-a transakcija
PoA: dokaz dostupnosti (dodaje se naknadno)
}
```

Svaki validator vodi sopstvenu traku (lane) linearni lanac CAR-ova. Trake rastu nezavisno jedna od druge, ograničene samo opterećenjem i kapacitetom resursa.

3.3 Proof of Availability (PoA)

Da bi CAR bio validan, potrebno je sakupiti $f+1$ potpisa od različitih validatora. Pošto sistem ima $n = 3f+1$ validatora, od kojih je najviše f vizantijskih, $f+1$ potpisa garantuje da je bar jedan ispravan validator u posedu podataka.

PoA se formira agregacijom potpisa (npr. BLS agregacija), što rezultuje jednim malim potpisom od svega 100–200 bajtova, bez obzira na broj potpisnika.

3.4 TipCut – predlog za konsenzus

Umesto da glasa o celim blokovima transakcija, Autobahn glasa o TipCut-u – vektoru od n referenci na najnovije CAR-ove sa PoA iz svake trake:

```
struct TipCut {
    map<lane_id, CAR> cars // n referenci, ~1KB ukupno
}
```

Ovo je ključna inovacija: umesto da se prenose gigabajti podataka tokom konsenzusa, prenosi se svega ~1KB heševa.

4. Glavne faze algoritma

4.1 Faza diseminacije podataka

Svaki validator kontinuirano izvršava sledeći proces:

1. Sakuplja batch – transakcije iz mempool-a do vremenskog prozora (npr. 50ms) ili limita veličine (npr. 1MB)
2. Lanči – kreira novi CAR sa `prev\_hash` koji pokazuje na prethodni CAR u svojoj traci

3. Emituje – šalje CAR svim validatorima (gossip protokolom)
4. Prikuplja potpise – sakuplja $f+1$ parcijalnih potpisa od validatora
5. Formira PoA – agregira potpise u dokaz dostupnosti
6. Ažurira traku – dodaje CAR u svoju traku

// Pseudokod: diseminacija

```
procedure disseminate(validator_id):
  while true:
    batch = mempool.pop_batch()
    prev = my_lane.get_last_car().hash
    car = new CAR(id=++my_lane.counter,
                  lane_id=validator_id,
                  prev_hash=prev,
                  batch_hash=hash(batch))
    broadcast(car)
    signatures = []
    while len(signatures) < f + 1:
      msg = wait_for_any_signature()
      if verify_signature(msg) and msg.car_id == car.id:
        signatures.append(msg.signature)
    car.poa = aggregate_signatures(signatures, f+1)
    my_lane.append(car)
```

4.2 Faza konsenzusa – Fast Path

U idealnim uslovima (stabilna mreža, ispravan lider), Autobahn koristi Fast Path, samo 1.5 runde razmene poruka:

1. Lider šalje PREPARE – emituje TipCut sa referencama na CAR-ove koji imaju PoA
2. Validatori šalju PRE-VOTE – proveravaju da svaki CAR u TipCut-u ima validan PoA
3. Lider sakuplja SVIH n glasova – agregira ih u Fast Commit QC i odmah ga emituje

Čim validatori vide Fast Commit QC, TipCut je finalizovan i svi CAR-ovi u njemu i njihovi preci postaju trajno uređeni.

4.3 Faza konsenzusa – Slow Path i View Change

Kada lider ne dobije sve glasove na vreme, sistem prelazi na Slow Path:

1. Lider sakuplja bar $2f+1$ pre-glasova → formira P-QC (Prepare Quorum Certificate)
2. Lider šalje CONFIRM poruku
3. Validatori vraćaju CONFIRM-ACK
4. Lider sakuplja $2f+1$ potvrda → formira C-QC (Commit Quorum Certificate)

5. Emituje SLOW_COMMIT – finalizacija

Ako ni $2f+1$ glasova ne stigne, validatori šalju TIMEOUT poruke:

1. Kad novi lider sakupi $2f+1$ timeout poruka → formira Timeout Certificate (TC)
2. Svi prelaze u novi view
3. Proces kreće ispočetka sa novim liderom

Važno: Trake i dalje rade tokom View Change-a, novi lider odmah ima šta da stavi u prvi TipCut.

```
// Pseudokod: konsenzus - lider
procedure leader_propose(lider_id):
    tipcut = build_tipcut()
    broadcast(PREPARE, tipcut)
    votes = collect_votes(timer)

    if len(votes) == N:
        broadcast(FAST_COMMIT, aggregate(votes))
        return

    if len(votes) >= 2*f + 1:
        pqc = aggregate(votes)
        broadcast(CONFIRM, pqc)
        acks = collect_confirm_acks(timer)
        if len(acks) >= 2*f + 1:
            broadcast(SLOW_COMMIT, aggregate(acks))
            return

    initiate_view_change()
```

4.4 Finalizacija i izvršenje

Kada se TipCut finalizuje:

1. Proširivanje kroz lance – za svaku traku u TipCut-u, prati `prev\_hash` unazad do već izvršenog CAR-a
2. Determinističko sortiranje – primena fiksnog pravila (npr. redom po trakama, pa po visini)
3. Izvršenje transakcija – tek sada se proverava stanje (balans, uslovi)

```
procedure finalize(tipcut):
    all_cars = []
    for each lane in tipcut.cars:
        car = tipcut.cars[lane.id]
```

```
while car is not null:
    all_cars.append(car)
    car = get_car_by_hash(car.prev_hash)
ordered_cars = deterministic_sort(all_cars)
for each car in ordered_cars:
    for each tx in get_transactions(car.batch_hash):
        execute(tx)
commit_state()
```

5. Rezultati

Eksperimentalni rezultati pokazuju:

1. Propusna moć – Autobahn dostiže ~234K tx/s, izjednačavajući se sa Bullshark-om
2. Latencija – Autobahn smanjuje latenciju za 2.1x u odnosu na Bullshark (280ms vs 592ms), približavajući se HotStuff-u
3. Oporavak od blipova – Autobahn se trenutno oporavlja, bez hangovera, dok HotStuff ima hangovere koji traju i do 30% duže od samog blipa

7. Zaključak

Autobahn predstavlja značajan napredak u oblasti BFT konsenzusa. Njegove ključne prednosti su:

1. Razdvajanje diseminacije i konsenzusa – omogućava da podaci teku nezavisno od konsenzus logike
2. CAR i PoA mehanizam – osigurava dostupnost podataka uz minimalne troškove
3. TipCut predlozi – omogućavaju konsenzus o ogromnim količinama podataka uz prenos od svega ~1KB
4. Seamless oporavak – trenutni oporavak od blipova bez hangovera

Autobahn se već koristi u praksi: implementiran je u blockchain sistemu Sei Giga, što potvrđuje njegovu praktičnu primenljivost. Otvoreni izvorni kod i detaljna dokumentacija omogućavaju dalji razvoj i integraciju u različite distribuirane sisteme.